

VII: ¿Por qué el recorrido inorden debe incluir
paréntesis?
porque preservan el orden correcto de las opera-
ciones y evitan ambigüedades al interpretar la
expresión.

VIII: ¿Cuál es la diferencia entre este árbol y
un árbol binario de búsqueda?

Árbol de expresión almacena operadores y
operandos para representar operaciones, el
de búsqueda organiza datos siguiendo
las reglas de menor a la izquierda y
mayor a la derecha para facilitar la búsqueda.

Juan Pablo Bustamante Ntz

27 07 2026

Scribe

14.

i. ¿Por qué se extrae primero el hijo derecho?
porque la pila funciona con el LIFO, al ingresar el primero el hijo izquierdo y luego el derecho, se extrae el derecho primero ya que fue el último en entrar.

ii. ¿Qué almacena realmente la pila durante el proceso?
Almacena los nodos del árbol que aún deben procesarse.

iii. ¿Qué indica que al finalizar quedan dos elementos en la pila?
Indica un error en la expresión o en la construcción del árbol, ya que al finalizar debe quedar solo un nodo que es la raíz del árbol.

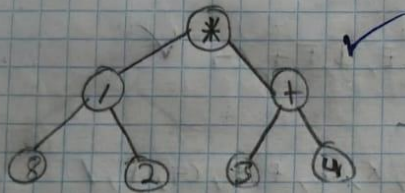
iv. ¿Qué sucede si aparece un operador cuando solo existe un nodo en la pila?
no se puede formar el subárbol porque faltan operandos.

v. ¿Qué recorrido produce la notación prefijo?
Preorden
Raíz → Izquierdo → Derecho

vi. ¿Qué recorrido produce la notación posfijo?
Postorden
Izquierdo → Derecho → Raíz

6. $82 / 34 (+ *) ((8 / 2) * (3 + 4))$

Token	Tipo	Acción Realizada	Contenido Pila
8	operando		(8)
2	operando		(2, 8)
/	operador		(8/2)
3	operando		(3, (8/2))
4	operando		(4, 3, (8/2))
+	operador		(3+4, (8/2))
*	operador		((8/2) * (3+4))



7. Pseudocódigo

✓ Función construir (expresión):
pila = nuevaPila()

Para cada token en expresión:

Si es operando:

nodo = nuevo Node Expression (token, nulo, nulo)
pila.push(nodo)

Si No

derecho = pila.pop()

izquierdo = pila.pop()

nodo = nuevo Node Expression (token, izquierdo, derecho)

pila.push(nodo)

← Retornar pila.pop()